

My Claude Usage Strategy

Personalized for: Professional | **Coder & Researcher** | Daily User | Deep Analysis

Powered by
Claude AI Expert

Professional

Coding / Technical

Learning / Research

Daily User

Deep Analysis

Recommended Primary Model

Claude Sonnet — Your Daily Driver

As a professional who codes daily and needs deep research, Sonnet hits the sweet spot of intelligence and speed. It handles complex technical reasoning, multi-step debugging, and thorough analysis — without the latency cost of Opus. You get substantive, accurate outputs fast enough to maintain a productive workflow.

Why This Model Fits You

- You write and debug code daily — Sonnet understands complex codebases, traces errors, and suggests refactors.
- Your research tasks need synthesis, not just retrieval — Sonnet connects concepts across sources intelligently.
- You need consistent quality without long wait times — Sonnet is significantly faster than Opus for daily use.
- As a daily user, cost and speed matter — Opus at scale is expensive; Sonnet delivers 80% of Opus quality at a fraction of the cost.
- Deep analysis is your core output type — Sonnet handles multi-step reasoning and structured analysis reliably.

Model Selection Guide

HAIKU	SONNET (Recommended)	OPUS
Fast & Light Quick lookups, syntax checks, short code explanations, rephrasing snippets. Use when speed matters more than depth.	Your Default Complex coding, debugging, research summaries, technical docs, architecture design. Best balance for your daily tasks.	Deep & Thorough Hard algorithmic problems, multi-system architecture, deep literature review, ambiguous research questions. Save for your hardest tasks.

Effort Level Guide

LOW	STANDARD (Default)	HIGH	MAX
Quick facts, simple syntax help, code formatting	Most daily coding & research tasks — your everyday default	Bug hunting, code review, research synthesis, system design	Critical architecture decisions, novel problem solving, complex proofs

Personalized Task Workflow

Task	Model	Effort	Reason
Quick syntax / API lookup	Haiku	Low	Factual, fast — no depth needed
Debugging code errors	Sonnet	High	Needs careful reasoning through the stack
Writing & reviewing code	Sonnet	Standard	Good intelligence for most code tasks
Researching a new tech/library	Sonnet	Standard	Thorough synthesis without over-spending tokens
System / architecture design	Opus	High	Long-horizon reasoning needed
Deep research analysis	Opus	Max	Maximum thoroughness for critical findings
Learning a new concept	Sonnet	Standard	Nuanced explanations without overkill
Writing technical docs	Sonnet	Standard	Clarity + structure is Sonnet's strength
Performance optimization	Opus	High	Requires deep trade-off analysis
Rephrasing / formatting output	Haiku	Low	Simple transformation — save credits

Personalized Daily Workflow

1 Morning Ramp-Up (Haiku + Low)

Start your day with quick lookups — API docs, syntax reminders, library comparisons. Use Haiku for anything you could Google. Save your Sonnet quota for real work.

2 Deep Coding Sessions (Sonnet + Standard/High)

When writing features, refactoring, or doing code reviews, use Sonnet at Standard. Bump to High when debugging — especially for tricky async issues, race conditions, or mysterious edge cases.

3 Research Blocks (Sonnet + Standard or Opus + High)

For learning a new library or framework, Sonnet + Standard is your go-to. When you need to deeply understand a paper, compare architectural approaches, or synthesize information across many sources — switch to Opus + High.

4 Architecture & Design (Opus + High/Max)

Reserve Opus for system-level decisions: microservices vs monolith, choosing a DB schema, designing APIs. These are high-stakes, low-frequency — worth the extra depth.

5 Documentation & Cleanup (Haiku + Low)

Writing JSDoc, README files, inline comments, or formatting code? Haiku handles these fast and cheaply. No need to burn Sonnet tokens on mechanical tasks.

6 End-of-Day Review (Sonnet + Standard)

Wrap up with Sonnet for summarizing what you built, writing commit messages, drafting PR descriptions, or planning tomorrow's tasks.

Prompting Tips for Coders & Researchers

For Coding

- Always share the error message AND the surrounding code — not just the error.
- Specify your language version and framework (e.g., Python 3.11, FastAPI 0.100).
- Tell Claude what you already tried — it avoids repeating those suggestions.
- Use: "Explain what this code does, then suggest improvements" for reviews.
- Ask for reasoning: "Walk me through why this approach is better."

For Research

- Start broad: 'Give me an overview of X, then focus on Y aspect.'
- Ask for comparisons: 'Compare approach A vs B for my use case: [context].'
- Request structure: 'Summarize in bullet points, then give a detailed explanation.'
- Use Opus for synthesis: 'What are the key trade-offs that experts disagree on?'
- Iterate: Start with Sonnet for a first pass, then Opus for the hard questions.

Biggest Mistakes to Avoid

- x Using Opus for every task** — It is slower and more expensive. Reserve it for genuinely hard problems. Sonnet handles 80% of your work equally well.
- x Running Low effort on debugging** — You will miss subtle root causes and waste time iterating. Always use High effort when diagnosing bugs.
- x Using Haiku for research synthesis** — Haiku lacks the depth to connect non-obvious concepts reliably. Sonnet is the minimum for any research task.
- x Skipping context in prompts** — Even Opus cannot help well without sufficient background on your codebase or research question. More context = better answers.
- x Using Max effort for routine tasks** — It adds latency and cost with no meaningful gain on simple queries. Match effort level to task complexity.
- x Treating all tasks the same** — Switching models and effort levels mid-workflow feels awkward at first but yields dramatically better results over time.

Final Recommendation

If you could use only ONE model + ONE effort level:

Claude Sonnet + Standard Effort

For a professional who codes and researches daily, Claude Sonnet at Standard effort covers approximately 80% of your work with excellent quality and speed. It handles complex code, nuanced explanations, and solid research synthesis — without the wait time of Opus.

Switch to Opus + High only when you hit a genuine wall on a hard problem. Switch to Haiku + Low only for simple lookups and text transformations.

This combination maximizes your output quality per unit of time and cost — the most important metric for a daily professional user.